

コマ 9: lvalue / rvalue + ポインタ

今日のゴール

`codegen()` と `codegen_lval()` を分離し、`&` と `*` を実装する。

第 08 回までの `codegen()` は、式を評価して値を `a0` に残す関数だった。しかし代入の左辺では、値ではなく「どこに書き込むか」というアドレスが必要になる。

この回では、式を 2 つの見方に分ける。

見方	意味	例
rvalue	式を評価して得られる値	<code>a</code> , <code>*p</code> , <code>1 + 2</code>
lvalue	書き込み先として使える場所	<code>a</code> , <code>*p</code>

この区別をコード上では次の 2 関数で表す。

```
codegen(node)      -> rvalue を計算し、値を a0 に置く
codegen_lval(node) -> lvalue のアドレスを計算し、アドレスを a0 に置く
```

1 この回で扱う範囲

対象にする機能は、単純なポインタの読み書きに限定する。

種類	例
アドレス取得	<code>&a</code>
間接参照	<code>*p</code>
ポインタ経由の書き込み	<code>*p = 20;</code>
ポインタ引数	<code>swap(&x, &y);</code>

この回では、ポインタも整数もすべて 8 バイト値として扱う。型サイズ、配列、ポインタ演算は第 10 回で扱う。

2 AST を確認する

まず、`&` と `*` を含むプログラムがどのような AST になるか確認する。

```
python3 scaffold/parse_viewer.py sessions/09_lvalue_rvalue/tests/deref_write.c
```

このプログラムの内容は次の通り。

```
int main() {
    int a;
    int *p;
    a = 10;
    p = &a;
    *p = 20;
    return a;
}
```

実行すると、次のような S 式が表示される。

```
(program
 (funcdef "main" :type int (params)
  (block (decl "a" :type int)
   (decl "p" :type (ptr int))
   (exprstmt
    (assign (var "a") (num 10)))
   (exprstmt
    (assign (var "p")
     (addr (var "a"))))
   (exprstmt
    (assign
     (deref (var "p"))
     (num 20)))
   (return (var "a")))))
```

注目するノードは次の 2 つである。

S 式	Node での表現	意味
(addr X)	ND_ADDR, node.operand == X	&X
(deref X)	ND_DEREF, node.operand == X	*X

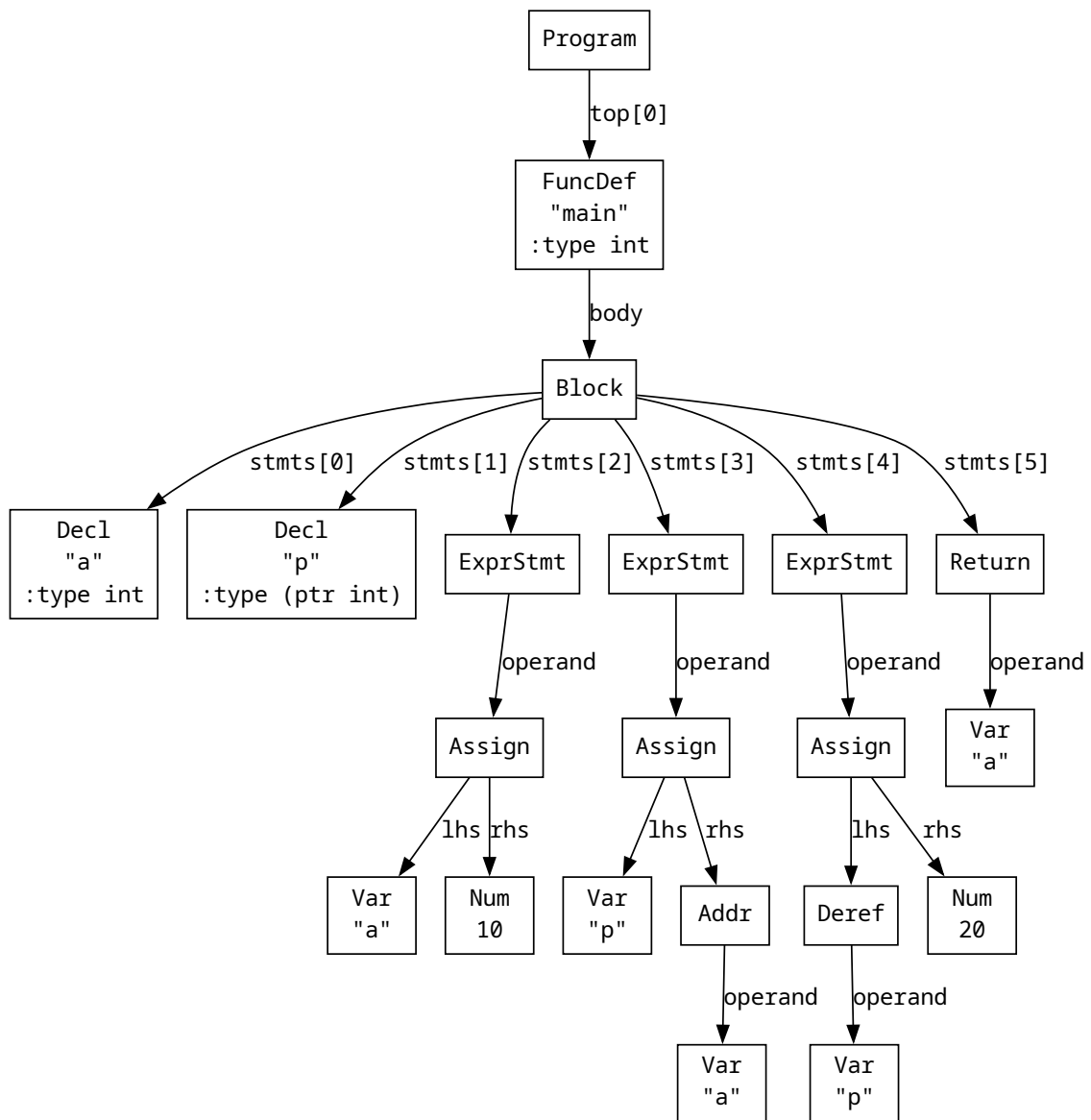
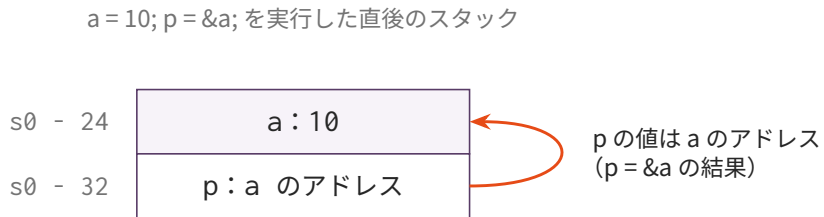


図1 deref_write.c のAST

`p = &a;` は、右辺に `(addr (var "a"))` を持つ代入である。`*p = 20;` は、左辺に `(deref (var "p"))` を持つ代入である。

3 rvalue と lvalue

同じ式でも、使われる場所によって意味が変わる。



*p = 20; は矢印の先 (= a のスロット) に 20 を書き込む

値がほしいとき (rvalue) → codegen() : a は 10、*p も 10
 アドレスがほしいとき (lvalue) → codegen_lval() : a は s0 - 24、*p は p の値 (矢印の先)

図 2 p = &a; 実行後のメモリと、rvalue / lvalue の関係

```
a = 10;
return a;
```

代入の左辺 a は「書き込み先」である。一方、return a; の a は「値を読む式」である。

つまり、a には 2 つの使い方がある。

C コード	必要なもの	処理
a = 10 の a	a のアドレス	codegen_lval(var a)
return a の a	a の値	codegen_lval(var a) の後に ld

この違いを明確にするために、codegen() と codegen_lval() を分ける。

p のスロットには a のアドレスが値として入っている。*p を lvalue として使うときは、この値 (矢印の先) がそのまま書き込み先アドレスになる。

4 codegen_lval() の役割

codegen_lval(node) は、代入先として使える式のアドレスを a0 に入れる。

変数 a の場合、スタック上のアドレスを計算する。

```
def codegen_lval(node):
    if node.kind == ND_VAR:
        offset = self._locals.get(node.name)
        if offset is None:
            raise RuntimeError(f"未定義の変数: '{node.name}'")
        self.emit(f"    addi a0, s0, {offset}")
        return
```

`*p` の場合は、`p` の値そのものが書き込み先アドレスである。

```
if node.kind == ND_DEREF:
    self.codegen(node.operand)
    return
```

`*p = 20;` では、まず `p` を `rvalue` として評価する。その結果、`a0` には `p` が指している先のアドレスが入る。このアドレスがそのまま左辺の `lvalue` になる。

5 & のコード生成

`&a` は「`a` のアドレスを値として得る」式である。したがって、`&` の中身を `lvalue` として評価すればよい。

```
if node.kind == ND_ADDR:
    self.codegen_lval(node.operand)
    return
```

`p = &a;` の流れは次のようになる。

1. 左辺 `p` のアドレスを `codegen_lval(var p)` で求める
2. 右辺 `&a` を `codegen(addr(var a))` で求める
3. `a` のアドレスを `p` のスロットに `sd` する

6 * のコード生成

`*p` は、`rvalue` として読む場合と `lvalue` として使う場合で処理が違う。

C コード	意味	処理
<code>return *p;</code>	<code>p</code> の指す先の値を読む	<code>codegen(p)</code> の後に <code>ld a0, 0(a0)</code>
<code>*p = 20;</code>	<code>p</code> の指す先に書き込む	<code>codegen_lval(deref p)</code> でアドレスだけ作る

rvalue としての `*p` は次のように生成する。

```
if node.kind == ND_DEREF:
    self.codegen(node.operand)
    self.emit(" ld a0, 0(a0)")
    return
```

7 代入のコード生成

代入 `lhs = rhs` では、左辺は lvalue、右辺は rvalue として扱う。

```
if node.kind == ND_ASSIGN:
    self.codegen_lval(node.lhs)    # 書き込み先アドレス
    push a0
    self.codegen(node.rhs)         # 書き込む値
    pop a1
    self.emit(" sd a0, 0(a1)")
    return
```

ここで重要なのは、左辺には `codegen()` ではなく `codegen_lval()` を使うことである。

8 関数引数としてのポインタ

第 08 回で関数呼び出しを実装しているため、ポインタを引数として渡す仕組み自体は新しい。`&x` を評価した結果はアドレス値なので、その値を通常の引数と同じように `a0` や `a1` に渡せばよい。

```

int swap(int *a, int *b) {
    int tmp;
    tmp = *a;
    *a = *b;
    *b = tmp;
    return 0;
}

int main() {
    int x;
    int y;
    x = 3;
    y = 7;
    swap(&x, &y);
    return x + y * 10;
}

```

`swap(&x, &y)` では、`&x` と `&y` がそれぞれアドレス値として評価され、関数に渡される。関数側では `*a` と `*b` により、呼び出し元の `x` と `y` を読み書きできる。

9 編集するファイル

- `mycc.py`

`importlib` でコマ 08 の `Codegen08` を継承した `Codegen09` に、以下の機能を追加する (スケルトンにあらかじめ書かれている)。

ハンドラメソッド

変更内容

`codegen_lval_Var(node)`

変数のアドレスを
`self._locals` から引いて
`a0` に返す

`codegen_lval_Deref(node)`

`self.codegen(node.operand)`
でアドレスを得る

`codegen(node)` の `match` 節に `'Addr'`

`self.codegen_lval(node.operand)`
でアドレスを値として返す

ハンドラメソッド

変更内容

`codegen(node)` の `match` 節に `'Deref'`

```
self.codegen(node.operand)
    の後
```

```
self.emit(" ld a0, 0(a0)")
```

`codegen(node)` の `match` 節に `'Assign'`左辺は `lval`、右辺は `rval` で評価し

```
self.emit(" sd a0, 0(a1)")
```

10 実装手順

1. スケルトンの `Codegen09` が `Codegen08` を `importlib` で継承していることを確認する
2. `codegen_lval(node)` に `'Deref'` handler を追加する
3. `codegen(node)` に `'Addr'` handler を追加する
4. `codegen(node)` に `rvalue` としての `'Deref'` handler を追加する
5. `codegen(node)` の `'Assign'` handler が左辺に `self.codegen_lval()` を使っていることを確認する
6. `swap.c` まで通ることを確認する

11 テスト

```
python3 scaffold/test_runner.py sessions/09_lvalue_rvalue
```

この回の主要テストは次の通り。

テスト	内容	期待値
<code>deref_read.c</code>	<code>p = &x; return *p;</code>	42
<code>deref_write.c</code>	<code>*p = 20; return a;</code>	20
<code>ptr_ops.c</code>	ポインタ先の値を読み書きする	55
<code>swap.c</code>	ポインタ引数で値を入れ替える	37